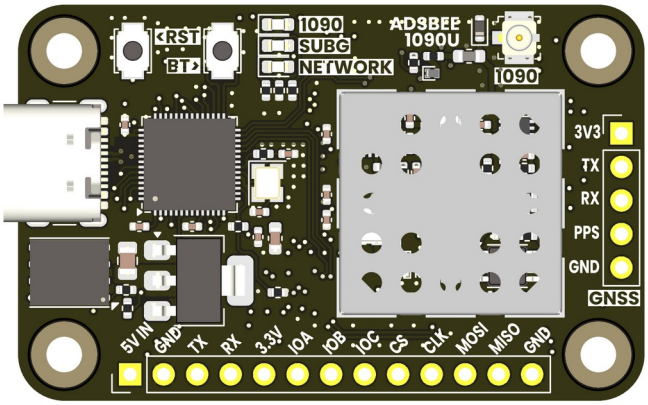
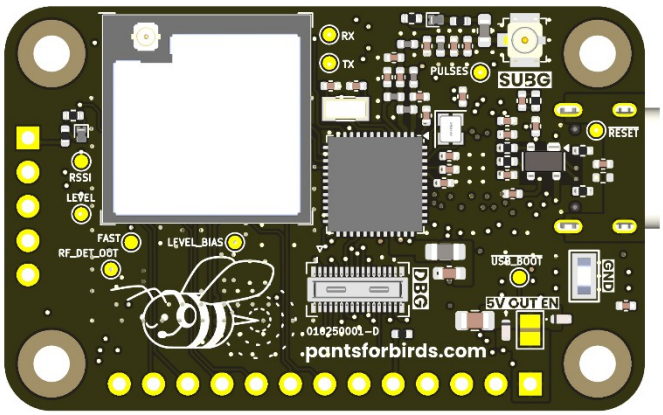
ADSBee 1090

Open Source Embedded ADS-B Receiver





Features

- 1090MHz Mode S and ADS-B packet decoding.
- Adjustable receive gain and trigger levels for customized tuning in diverse RF environments.
- Tunable sub-GHz transceiver for UAT / additional protocols.
- Multiple output formats over UART or USB: Raw Frames / ADSBee CSV / MAVLINK1 / MAVLINK2 / Mode S Beast / GDL90
- GNSS module input (UART + PPS) for MLAT or Remote ID applications.
- 2.4GHz 802.11 module for connecting directly to ADS-B databases via WiFi or broadcasting Remote ID beacon frames in UAS applications.
- Integrated M2.5 mounting holes.
- Optional PoE pant for power + data over Ethernet.
- Firmware updates over USB, Serial, WiFi, Ethernet.

Applications

- Aircraft detection for robotics and embedded projects.
- Electronic Flight Bag ADS-B receiver.
- Standalone feeder device for online ADS-B databases. No external compute required, just add power and WiFi!

Quick Specs

Supply Voltage	5V (via USB or 5V pin)
Supply Current	120mA (WiFi disabled) ~400mA (WiFi enabled)
Minimum RF Input Power Level	-85dBm
Simultaneous Aircraft Tracks Supported	≤200
Connectors	1090MHz RF In: U.FL / MHF1 Sub-GHz RF In/Out: U.FL / MHF1 802.11 RF Out: W.FL / MHF3 Power / Data: USB C GPIO / UART: 0.1” Pin Headers

Open Source Hardware + Software

Github Repository: https://github.com/coolnamesalltaken/adsbee

All hardware schematics and source code files required to build ADSBee 1090 are available under a GNU GPL v3 license. This means that they can be freely incorporated into other open-source projects that utilize a compatible license. The hope is that by opening the design to contributions and feedback from a community of users, the functionality of the ADSBee 1090 will be enhanced over time.

Note that the GPL v3 license applies to design and source code files, and not devices. ADSBee 1090 units purchased from Pants for Birds may be used in commercial applications without any licensing restrictions.

For commercial licensing requests of ADSBee hardware or software design files, please contact john@pantsforbirds.com.

Contents

Features....1

Applications....1

Quick Specs....1

Open Source Hardware + Software....1

Revision History....6

1 Background....7

2 Communication Interfaces....8

2.1 Console Interface....8

2.2 COMMS_UART Interface....8

2.3 GNSS_UART Interface....8

3 Status Indicators....9

4 AT Commands....10

4.1 AT+BAUD_RATE....10

4.1.1 Set Baud Rate....10

4.1.2 Query Baud Rate....10

4.2 AT+BIAS_TEE_ENABLE....11

4.2.1 Turn the Bias Tees On or Off....11

4.2.2 Query the Status of the Bias Tee....11

4.3 AT+BOOT_USB_UF2....11

4.4 AT+CONFIG_HASH....12

4.5 AT+DEVICE_INFO....12

4.6 AT+ESP32_ENABLE....13

4.6.1 Turn the ESP32 On or Off....13

4.6.2 Query the Status of the ESP32....13

4.7 AT+ESP32_FLASH....13

4.8 AT+ETHERNET....13

4.8.1 Enable or Disable Ethernet....13

4.8.2 Query Ethernet Setting....14

4.9 AT+FEED....14

4.9.1 Configure a Feed....14

4.9.2 Query Configuration of All Feeds....15

4.9.3 Query Configuration of a Single Feed....15

4.10 AT+FEED_ENABLE....15

4.10.1 Enable or Disable All Feeds....15

4.10.2 Query the Feed Enable Setting....15
4.11 AT+GNSS....16
4.11.1 Configure GNSS....16
4.11.2 Query GNSS Configuration....16
4.12 AT+GNSS_FIX....16
4.12.1 Unsolicited AT+GNSS_FIX Messages....16
4.12.2 Query GNSS Fix....16
4.13 AT+HELP....17
4.14 AT+HOSTNAME....17
4.14.1 Set the Device Hostname....17
4.14.2 Query the Device Hostname....17
4.15 AT+LED_BLINK....18
4.16 AT+LED_ENABLE....19
4.16.1 Enable or Disable the Status Indicators....19
4.16.2 Query the Status Indicator Enable Setting....19
4.17 AT+MAVLINK_ID....20
4.17.1 Set the MAVLINK System ID and Component ID....20
4.17.2 Query the MAVLINK System ID and Component ID....20
4.18 AT+LOG_LEVEL....20
4.19 AT+PROTOCOL_OUT....21
4.19.1 Set the Reporting Protocol for a Serial Interface....21
4.19.2 Query the Reporting Protocol for All Serial Interfaces....21
4.20 AT+REBOOT....21
4.21 AT+RX_ENABLE....22
4.21.1 Enable or Disable the Receiver(s)....22
4.21.2 Check Whether the Receiver(s) are Currently Enabled....22
4.22 AT+RX_POSITION....23
4.22.1 Query Receiver Position....23
4.22.2 Set Receiver Position....24
4.23 AT+SETTINGS....25
4.23.1 Query Current Settings....25
4.23.2 Dump Settings in AT Format....26
4.23.3 Load Settings from EEPROM....26
4.23.4 Save Settings to EEPROM....26
4.23.5 Reset Settings to Factory Default....26
4.24 AT+SUBG_ENABLE....27

4.24.1 Set Sub-GHz Receiver Enable Status and Line Drive Type....27
4.24.2 Query Sub-GHz Receiver Enable Status and Line Drive Type....27
4.25 AT+TEST....27
4.26 AT+TL_READ....28
4.27 AT+TL_OFFSET....28
4.27.1 Set Trigger Level....28
4.27.2 Query Trigger Level....28
4.28 AT+UPTIME....29
4.29 AT+WATCHDOG....29
4.29.1 Configure the Watchdog Timer....29
4.29.2 Query the Watchdog Timer....29
4.29.3 Test the Watchdog Timer....29
4.30 AT+WIFI_AP....30
4.30.1 Configure the WiFi Access Point....30
4.30.2 Query the WiFi Access Point Configuration....30
4.31 AT+WIFI_STA....31
4.31.1 Configure the WiFi Station....31
4.31.2 Query the WiFi Station Configuration....31
5 Reporting Protocols....32
5.1 CSBee....33
5.1.1 CSBee Protocol Version History....33
5.1.2 Mode S Aircraft Message....34
5.1.3 UAT Aircraft Message....41
5.1.4 Statistics Message....45
5.2 GDL90....46
5.3 MAVLINK....47
5.3.1 MAVLINK_HEARTBEAT （ Message ID 0 ） Packet Definition....47
5.3.2 MAVLINK ADSB_VEHICLE (Message ID 246) Packet Definition....47
5.3.3 MAVLINK_REQUEST_DATA_STREAM (Message ID 66) Packet Definition....48
5.3.4 MAVLINK MESSAGE_INTERVAL (Message ID 244) Packet Definition....48
5.4 Mode S Beast....49
5.4.1 Beast: Mode S Frames....49
5.4.2 Beast: UAT Frames....49
5.5 Raw Packets....50
5.5.1 Raw Mode S Frames....50

5.5.2 Raw UAT ADSB Frames....50
5.5.3 Raw UAT Uplink Frames....51
5.6 Aircraft JSON WebSocket....52
5.6.1 Message Format....52
5.6.2 JSON Fields....53

Revision History

20250719

- Initial release with version control.

20250821

- Update CSBee protocol with new bit flag positions, new field for separate baro/GNSS vertical rates.
- Change AT+TL_SET AT command to AT+TL_OFFSET AT command.
- Change AT+PROTOCOL AT command to separate AT+PROTOCOL_IN and AT+PROTOCOL_OUT AT commands.

20260502

- Update CSBee protocol to version 3.
- Add new AT commands:
  - AT+BOOT_USB_UF2
  - AT+MAVLINK_ID
  - AT+RX_POSITION
  - AT+UPTIME

20261013

- Added new AT commands:
  - AT+GNSS
  - AT+GNSS_FIX
  - AT+LED_BLINK
  - AT+LED_ENABLE
  - AT+FEED_ENABLE
  - AT+CONFIG_HASH
- Added the Status Indicators section describing the 1090, SUBG, and NETWORK LEDs.
- Corrected the AT Commands introduction: command replies are not prefixed with “+”.

1 Background

ADSBee 1090 was created as an attempt to build a low-cost ADS-B receiver without the use of an FPGA (found in most embedded ADS-B receivers on the market) and without the need for external compute (a requirement of most SDR-based ADS-B receivers). ADSBee 1090 accomplishes this through the use of a low-cost dual core microcontroller (RP2040) with flexible IO peripherals (PIO) that run a set of custom-written programs for preamble detection and packet decoding. By dedicating the RP2040's PIO peripherals to the tasks required to find and decode ADS-B packets, ADSBee 1090 frees up its remaining cores to perform the logical functions required to validate checksums on decoded packets and decipher aircraft information (position, altitude, callsign, etc).

The ADSBee 1090 reports decoded aircraft information over UART and USB interfaces in a variety of protocols, and can utilize its attached ESP32 S3 module to host WiFi networks for data streaming to other devices, or connect to existing WiFi networks in order to upload data to the internet or other devices on the network. Ethernet connectivity (and 802.3af Power over Ethernet) is possible using an external Ethernet pant.

Dual band versions of the ADSBee 1090, such as the ADSBee 1090U, includes a TI CC1312 sub-GHz MCU for decoding additional radio protocols like UAT. This datasheet applies to all devices in the ADSBee family that run the ADSBee 1090 firmware (e.g. ADSBee 1090, ADSBee 1090U, ADSBee m1090, ADSBee GS3M PoE).

2 Communication Interfaces

2.1 Console Interface

The CONSOLE interface (USB-C connector) on the ADSBee 1090 can be used to supply the device with power, configure the device's internal parameters via AT commands, and receive data from the device.

No baud rate configuration is necessary for the CONSOLE interface. Note that AT commands must be suffixed with CR+LF ("\r\n") in order to be processed by the AT command parser.

2.2 COMMS_UART Interface

The COMMS_UART interface is used for data output and can support several different protocols. Data can be streamed out of the CONSOLE and COMMS_UART interfaces simultaneously. The baud rate of the COMMS_UART interface can be adjusted via AT commands.

COMMS_UART Parameters

Parameter	Value
Baud Rate	115200 baud (default)
Data Bits	8
Stop Bits	0 (N)
Parity Bits	1
Logic Level	3.3V

2.3 GNSS_UART Interface

The ADSBee 1090's GNSS module connector includes a UART interface which can be used to receive NMEA sentences from a GNSS module. The baud rate of the GNSS_UART interface can be adjusted via AT commands.

GNSS_UART Parameters

Parameter	Value
Baud Rate	9600 baud (default)
Data Bits	8
Stop Bits	0 (N)
Parity Bits	1
Logic Level	3.3V

3 Status Indicators

The ADSBee 1090 has three status indicator LEDs on its top side. Each LED is driven by a different processor on the board and reports activity for that processor only.

Indicator	Description	Normal Behavior
1090	Driven by the RP2040. Indicates 1090 MHz receive activity.	Flashes briefly each time a 1090 MHz packet is accepted into the aircraft dictionary. At startup, blinks once if the device is running from flash partition 0, or twice otherwise.
SUBG	Driven by the CC1312 sub-GHz receiver. Indicates UAT receive activity. Present only on dual band versions of the ADSBee 1090, such as the ADSBee 1090U.	Flashes briefly each time a UAT packet is decoded.
NETWORK	Driven by the ESP32. Indicates network and coprocessor activity.	Flashes briefly on each SPI transaction between the ESP32 and the RP2040.

All three indicators can be turned off with AT+LED_ENABLE=0. AT+LED_BLINK forces any indicator on for a fixed duration and ignores the AT+LED_ENABLE setting entirely, which makes it useful for verifying indicators on a production test fixture.

In firmware builds that enable the watchdog reboot warning, the 1090 indicator is also held on solid for approximately five seconds after a reboot caused by the watchdog.

4 AT Commands

AT Commands are used to configure the ADSBee 1090 receiver's internal parameters via the CONSOLE interface. AT commands can be used to set values (with the '=' operator, e.g. "AT+BAUD_RATE=COMMS_UART,115200") or query values (with the '?' operator, e.g. "AT+BAUD_RATE?").

AT commands and parameters must be all-caps. Whitespace is not ignored and should only be used intentionally (e.g. as part of a Wi-Fi network SSID which contains spaces). Arguments are separated by a single comma and no spaces.

All AT command arguments are optional. Arguments will be ignored if left as blank. For instance, to set the second parameter of AT+TL_SET to 3000 without changing the value of the first parameter, the command "AT+TL_SET=,3000" can be sent. Likewise, to change the first parameter to 2000 without changing the value of the second, the command "AT+TL_SET=2000," can be sent.

Commands from the user to the ADSBee 1090 are prefixed with "AT+" (e.g. "AT+BAUD_RATE..."). Replies may begin with the relevant command name, without the "AT+" prefix (e.g. "BAUD_RATE=...") or nothing (e.g. "OK"). Replies to AT command queries may include text enclosed by parentheses (e.g. "BAUD_RATE=115200(COMMS),9600(GNSS)". These text blobs are annotations to improve human readability of the reply.

IMPORTANT NOTE

No settings will persist after shutdown unless `AT+SETTINGS=SAVE` is executed to write new settings values to non-volatile storage.

4.1 AT+BAUD_RATE

4.1.1 Set Baud Rate

```
AT+BAUD_RATE=<iface>,<baud_rate>
```

Set the baud rate for a particular serial interface. Note that the USB C port (CONSOLE interface) is a virtual COM port and does not have a baud rate that can be changed.

Parameter	Description	Values
iface	Serial interface to set the baud rate for.	COMMS_UART, GNSS
baud_rate	Baud rate, in characters per second.	9600-115200

4.1.2 Query Baud Rate

```
AT+BAUD_RATE?
+BAUD_RATE=<comms_uart_baud_rate>(COMMS_UART),<gnss_uart_baud_rate>(GNSS_UART)
```

Queries the baud rate for the COMMS_UART and GNSS_UART interfaces. Note that the baud rates returned by this query are the actual system baud rates, which should be close to the baud rates that were requested with AT+BAUDRATE but may be slightly different due to the particulars of how baud rates are implemented inside the RP2040.

Parameter	Description	Values
comms_uart_baud_rate	Baud rate on the COMMS_UART interface.	9600-115200
gnss_uart_baud_rate	Baud rate on the GNSS_UART interface.	9600-115200

4.2 AT+BIAS_TEE_ENABLE

4.2.1 Turn the Bias Tees On or Off

```
AT+BIAS_TEE_ENABLE=<1090_bt_enabled>,<subg_bt_enabled>
```

Enable or disable the bias tee to enable use of an external LNA.

Parameter	Description	Acceptable Values
1090_bt_enabled	Whether the 1090MHz bias tee is enabled (3.3VDC supplied to the 1090 RF connector) or disabled (no DC voltage supplied to the 1090 RF connector).	0 – Bias tee is disabled. 1 – Bias tee is enabled.
subg_bt_enabled	Whether the Sub-GHz radio bias tee is enabled (3.3VDC supplied to the SubG RF connector) or disabled (no DC voltage supplied to the SubG RF connector).	0 – Bias tee is disabled. 1 – Bias tee is enabled.

4.2.2 Query the Status of the Bias Tee

```
AT+BIAS_TEE_ENABLE?
+BIAS_TEE_ENABLE=<1090_bt_enabled>,<subg_bt_enabled>
```

Queries the status of the bias tee. See table in 4.1.1 for values and their meanings.

4.3 AT+BOOT_USB_UF2

Reboots the ADSBee into UF2 Bootloader mode. ONLY use this command if you have USB access to your ADSBee! There is no way to recover from the factory UF2 bootloader other than resetting the device or flashing a firmware file to it over USB.

```
AT+BOOT_USB_UF2=<reboot_passcode>
```

Parameter	Description	Acceptable Values
reboot_passcode	A passcode that makes it harder to run this command by accident.	“1DEADBEE”

4.4 AT+CONFIG_HASH

Prints hashes of the settings, the CC1312 CCFG (customer configuration) and the firmware images, so a receiver can be compared with a reference receiver in one line. Two receivers with the same firmware and the same settings print the same values. An example query and reply is shown below.

```
> AT+CONFIG_HASH?
CONFIG_HASH=0x3D7A91C4
SETTINGS_FLEET=0x9140FC1E (settings version 18, 1340 Bytes)
SETTINGS_FULL=0x2F6B0D55
CCFG_CC1312=0x4BB06FBD
FIRMWARE=0x4D476C34
FIRMWARE_RP2040=0x7DC53693 (UF2 image, partition 0, 2950400 Bytes)
FIRMWARE_ESP32=0xC90EA4DB
FIRMWARE_CC1312=0x6F59BCF1
```

Line	Description
CONFIG_HASH	Hash of SETTINGS_FLEET, CCFG_CC1312 and FIRMWARE. Compare this first.
SETTINGS_FLEET	All settings except the hostname, WiFi access point SSID and channel, feed receiver IDs, Remote ID UAS ID and receiver position values.
SETTINGS_FULL	All settings.
CCFG_CC1312	The CCFG of the CC1312, read from its flash at boot.
FIRMWARE	Hash of the three FIRMWARE_<processor> lines.
FIRMWARE_RP2040	The application image in the running flash partition. The .uf2 image and the .ota images for partition 0 and partition 1 each have their own value.
FIRMWARE_ESP32	The first 4 bytes of the SHA-256 that the ESP32 build stores in its application image.
FIRMWARE_CC1312	The application image in the CC1312's flash.

A line reads UNKNOWN when a processor is off or did not answer. Releases list the firmware and CCFG values in adsbee_1090-<version>.hashes.txt.

4.5 AT+DEVICE_INFO

Queries the device information of the ADSBee. An example query and reply is shown below.

```
> AT+DEVICE_INFO?
Part Code: 010240002F-20240907-000001
RP2040 Firmware Version: 0.6.2
RP2040 Firmware Build: Release
OTA Key 0:
cf737c4fce892fab44c6ec0d5d0f66dccf06c5630ac5b531f4067599df014d5ef213bb73205a9ce37
8318f259e3d01e3b99c089a75ac0735b204256b9371fbce
OTA Key 1:
1701a736dc40c837f318c9e7ae82557a39fe07588d3745602bb9269b13843b87ca7962daecc9897c6
dfb0ac4ec77997e9cc34cd356e415fc46eb4ae3dd42b717
ESP32 Firmware Version: 0.6.2
ESP32 Base MAC Address: 64:E8:33:5D:0B:C4
ESP32 WiFi Station MAC Address: 64:E8:33:5D:0B:C4
ESP32 WiFi AP MAC Address: 64:E8:33:5D:0B:C5
ESP32 Bluetooth MAC Address: 64:E8:33:5D:0B:C6
ESP32 Ethernet MAC Address: 64:E8:33:5D:0B:C7
```

The RP2040 Firmware Build line reads Debug or Release (firmware 0.9.1-rc7 and later). Published releases are Release builds; build.sh -d in firmware/adsbee_1090 makes a Debug build.

4.6 AT+ESP32_ENABLE

4.6.1 Turn the ESP32 On or Off

```
AT+ESP32_ENABLE=<enabled>
```

Turns the ESP32 on or off using its enable line. When the ESP32 is turned off, the ADSBee 1090 will draw less power, but the WiFi and network capabilities will be unavailable. This operational mode is useful for conserving energy in applications where the ADSBee is only being communicated with via its serial interfaces.

Parameter	Description	Acceptable Values
enabled	Whether the ESP32 is turned on.	0 – ESP32 is turned on. 1 – ESP32 is turned off.

4.6.2 Query the Status of the ESP32

```
AT+ESP32_ENABLE?
ESP32_ENABLE=<enabled>
```

Queries whether the ESP32 is turned on or off. See table in 4.6.1 for the values and their meanings.

4.7 AT+ESP32_FLASH

```
AT+ESP32_FLASH
```

Flashes the ESP32 with the firmware image stored on the RP2040. This should never be needed, since the RP2040 does a firmware version check of the ESP32 on startup and automatically reflashes he firmware if it is out of date. The AT+ESP32_FLASH command is only used in weird debugging scenarios where the version number of the firmware on the ESP32 matches the version of the firmware stored on the RP2040, but the firmware stored on the RP2040 is different and needs to be flashed onto the ESP32.

4.8 AT+ETHERNET

4.8.1 Enable or Disable Ethernet

Enables or disables Ethernet functionality on the ESP32. Must be connected to a W5500 Ethernet IC via SPI.

```
AT+ETHERNET=<enabled>
```

Parameter	Description	Acceptable Values
enabled	Whether Ethernet connectivity is enabled via the ESP32 and a connected W5500 Ethernet IC.	0 – Ethernet is enabled. 1 – Ethernet is disabled.

4.8.2 Query Ethernet Setting

```
AT+ETHERNET?
ETHERNET=1
```

4.9 AT+FEED

4.9.1 Configure a Feed

```
AT+FEED=<index>,<uri>,<port>,<active>,<protocol>
```

Example command for configuring feed 0 to send data to airplanes.live, which accepts Mode S Beast protocol on port 30004 at the static IP 78.46.234.18:

```
AT+FEED=0,78.46.234.18,30004,1,BEAST
```

Parameter	Description	Acceptable Values
index	Which feed slot to configure.	0-5
uri	Static IP address or hostname of the feed destination. If the field is given as a hostname, a DNS lookup will be performed in order to convert the hostname to an IP address.	Public IP address, e.g. 78.46.234.18 Hostname, e.g. feed.whereplane.xyz Maximum length 64 characters.
port	Port number to feed to.	0-65535
active	Whether the feed should be activated. This can be used to temporarily disable a feed while still keeping its configuration information intact.	0 – Feed is active. 1 – Feed is disabled.
protocol	Which protocol to use when feeding.	BEAST – Send validated packets in Mode S Beast format. BEAST_RAW – Send both validated and invalid (bad checksum) packets in Mode S Beast format.

4.9.2 Query Configuration of All Feeds

```
AT+FEED?
FEED=0(INDEX),192.168.1.103(URI),30004(PORT),1(ACTIVE),BEAST(PROTOCOL),bee00038172d18c(RECEIVER_ID)
FEED=1(INDEX),(URI),0(PORT),0(ACTIVE),NONE(PROTOCOL),bee00038172d18c(RECEIVER_ID)
FEED=2(INDEX),(URI),0(PORT),0(ACTIVE),NONE(PROTOCOL),bee00038172d18c(RECEIVER_ID)
FEED=3(INDEX),(URI),0(PORT),0(ACTIVE),NONE(PROTOCOL),bee00038172d18c(RECEIVER_ID)
FEED=4(INDEX),78.46.234.18(URI),30004(PORT),1(ACTIVE),BEAST(PROTOCOL),bee00038172d18c(RECEIVER_ID)
FEED=5(INDEX),feed.whereplane.xyz(URI),30004(PORT),0(ACTIVE),BEAST(PROTOCOL),bee00038172d18c(RECEIVER_ID)
```

4.9.3 Query Configuration of a Single Feed

```
AT+FEED?<index>
```

Example query for configuration of feed 0 only:

```
AT+FEED?0
FEED=0(INDEX),192.168.1.103(URI),30004(PORT),1(ACTIVE),BEAST(PROTOCOL),bee00038172d18c(RECEIVER_ID)
```

4.10 AT+FEED_ENABLE

4.10.1 Enable or Disable All Feeds

```
AT+FEED_ENABLE=<enabled>
OK
```

Master switch for all outbound network feeds configured with AT+FEED. When disabled, no feed connections are opened and any open feed connections are closed, regardless of whether individual feeds are marked as active. The per-feed active flags set with AT+FEED are not modified, so re-enabling feeds with AT+FEED_ENABLE=1 resumes exactly the feeds that were previously active. The setting is synchronized to the ESP32 coprocessor. Feeds are enabled by default.

Parameter	Description	Acceptable Values
enabled	Whether outbound network feeds are enabled.	0 – All feeds are disabled. 1 – Feeds marked as active with AT+FEED are enabled (default).

4.10.2 Query the Feed Enable Setting

```
AT+FEED_ENABLE?
FEED_ENABLE=<enabled>
```

Queries whether outbound network feeds are enabled. This setting does not persist across a power cycle unless AT+SETTINGS=SAVE is run to write it to non-volatile storage.

4.11 AT+GNSS

4.11.1 Configure GNSS

```
AT+GNSS=<enabled>,[type],[notify]
```

Parameter	Description	Acceptable Values
enabled	Whether the GNSS system should be activated.	0-1
type	Optionally specifies the style of messages that are parsed by the GNSS system. **NONE** – default value, will disable the GNSS system **GENERIC** – parses NEMA 0138 style messages **UBX_MIA** – parses uBlox MIA style messages, including manufacturer specific messages.	NONE, GENERIC, UBX_MIA
notify	Optionally specifies whether the GNSS system should send an unsolicited **AT+GNSS_FIX** message when the fix status changes state from invalid to valid, or valid to invalid.	0-1

4.11.2 Query GNSS Configuration

```
AT+GNSS?
GNSS=1,GENERIC,1
```

4.12 AT+GNSS_FIX

4.12.1 Unsolicited AT+GNSS_FIX Messages

When enabled, the ADSBee sends unsolicited AT+GNSS_FIX messages on the COMMS_UART and web console. These messages are emitted when the state of the GNSS_FIX changes from valid to invalid, or invalid to valid. The maximum transmission rate of these messages is once per second.

4.12.2 Query GNSS Fix

```
AT+GNSS_FIX?
GNSS_FIX=<valid>,<lat>,<lon>,<alt_ft>,<heading>,<speed_kts>,<sats>,<UTC>,<pps_count>
GNSS_FIX=1,39.752150,-122.205914,719,324.6,0,7,01:40:01,1530
```

Parameter	Description	Expected Values
valid	Whether or not the GNSS system has a valid position fix	0-1
lat	GNSS position latitude	-90.000 to +90.000 degrees
lon	GNSS position longitude	-180.000 to +180.000 degrees
alt_ft	GNSS altitude	0-50,000 feet
heading	GNSS track heading	0.0-360.0 degrees
speed_kts	GNSS track speed	0-999 knots
sats	GNSS satellites in view	0-128

UTC	Current UTC time in HH:MM:SS	00:00:00-23:59:59
pps_count	Count of pulse-per-second pulses counted by adsbee from the external GNSS module	0-4,294,967,295

4.13 AT+HELP

```
AT+HELP
AT Command Help Menu:
BAUDRATE:
        AT+BAUDRATE=<iface>,<baudrate>
        Set the baud rate of a serial interface.
        AT_BAUDRATE?
        Query the baud rate of all serial interfaces.
BIAS_TEE_ENABLE:
        AT+BIAS_TEE_ENABLE=<enabled>
        Enable or disable the bias tee.
        BIAS_TEE_ENABLE?
        Query the status of the bias tee.
...
```

Prints the available AT commands as well as how to use them. The list of commands in the preview above is abridged.

4.14 AT+HOSTNAME

4.14.1 Set the Device Hostname

```
AT+HOSTNAME=<hostname>
```

Sets the device hostname, which can make things easier when configuring a local network. Hostname is also used for mDNS, allowing you to visit the device's webpage at <hostname>.local.

4.14.2 Query the Device Hostname

```
AT+HOSTNAME?
HOSTNAME=ADSBee1090-0240907000001
```

4.15 AT+LED_BLINK

```
AT+LED_BLINK=<duration_ms>,<led>[,<led>...]
OK
```

Forces one or more status indicator LEDs solid on for duration_ms milliseconds, then turns them off again. The command is non-blocking and returns immediately. Up to three led arguments may be supplied; repeating the same led has no additional effect. This command ignores the AT+LED_ENABLE setting and lights the requested indicators even when the LEDs are disabled, which makes it useful for test fixtures and production test. See section 3 for a description of each indicator.

Parameter	Description	Acceptable Values
duration_ms	Number of milliseconds that the requested indicators are held on.	1 – 60000
led	Indicator to light. May be repeated up to three times to light more than one indicator.	1090 – The 1090 MHz activity indicator. SUBG – The sub-GHz (UAT) activity indicator. NETWORK – The network activity indicator. ALL – All three indicators.

All requested indicators must accept the request for the command to succeed. If a target coprocessor is disabled or unreachable, the command replies with an error that names the indicators that failed; indicators that already started blinking are not turned back off.

```
ERROR Duration must be 1-60000 ms.
ERROR Invalid LED 'X'. Must be 1090, SUBG, NETWORK, or ALL.
ERROR Failed to blink: SUBG(disabled).
```

A blink request does not shorten a longer blink that is already in progress on the same indicator.

4.16 AT+LED_ENABLE

4.16.1 Enable or Disable the Status Indicators

```
AT+LED_ENABLE=<enabled>
OK
```

Enables or disables the hardware status indicator LEDs. The setting applies to all three indicators and is synchronized to the ESP32 and CC1312 coprocessors. Disabling the indicators turns the 1090 indicator off immediately rather than waiting for the current flash to expire. Note that AT+LED_BLINK overrides this setting.

Parameter	Description	Acceptable Values
enabled	Whether the status indicator LEDs are enabled.	0 – Status indicators are disabled. 1 – Status indicators are enabled.

4.16.2 Query the Status Indicator Enable Setting

```
AT+LED_ENABLE?
LED_ENABLE=<enabled>
```

Queries whether the status indicator LEDs are enabled. This setting does not persist across a power cycle unless AT+SETTINGS=SAVE is run to write it to non-volatile storage.

4.17 AT+MAVLINK_ID

MAVLINK components have a system and component ID. For more information about system and component IDs, visit: https://mavlink.io/en/services/mavlink_id_assignment.html.

4.17.1 Set the MAVLINK System ID and Component ID

```
AT+MAVLINK_ID=<system_id>,<component_id>
OK
```

Parameter	Description	Acceptable Values
system_id	Unique identifier for the MAVLINK system (e.g. the aircraft). All components in the system should have the same system_id.	1-255
component_id	Unique identifier for a component within the MAVLINK system (e.g. the ADSBee). All components in the system should have a unique component_id.	1-255 Default: 156 (MAV_COMP_ID_ADSB as defined here: https://mavlink.io/en/messages/common.html#MAV_COMPONENT)

4.17.2 Query the MAVLINK System ID and Component ID

```
AT+MAVLINK_ID?
System ID: 1
Component ID: 156
```

4.18 AT+LOG_LEVEL

```
AT+LOG_LEVEL=<log_level>
```

Parameter	Description	Acceptable Values
log_level	Log verbosity level for the console. Anything being printed with a lower verbosity than the specified level will be suppressed. A verbosity level of AT+LOG_LEVEL=WARNINGS is recommended. Terminal prints are color coded on the Serial CLI (but not on the Web CLI). Errors are printed in red, warnings are printed in yellow, and other prints have no color coding.	INFO – Informational logs. Details every packet received by the ADSBee and whether it passed checksum. Also prints all warnings and errors. WARNINGS – Warning logs. Prints out every time something unexpected happens. Warnings include recoverable errors and unexpected values encountered within ADS-B packets being decoded. Also prints all errors. ERRORS – Error logs. Prints every time there is a severe error, such as packets being lost in communication between the RP2040 and ESP32, a peripheral failing to communicate properly, or fatal errors in code. SILENT – No logs. Only AT command replies and console protocols will be printed. Use this setting to avoid having log messages injected into the console text stream if it is

		being used to report a protocol to another device.

4.19 AT+PROTOCOL_OUT

4.19.1 Set the Reporting Protocol for a Serial Interface

```
AT+PROTOCOL_OUT=<iface>,<protocol>
```

Parameter	Description	Values
iface	Serial interface to set the baudrate for.	CONSOLE, COMMS_UART
protocol	Reporting protocol for the specified serial interface	NONE – No data reported. RAW – Raw plain text reporting protocol. BEAST – Mode S Beast binary reporting protocol. CSBEE – CSBEE plain text reporting protocol. MAVLINK1 – MAVLINK 1 binary reporting protocol. MAVLINK2 – MAVLINK 2 binary reporting protocol. GDL90 – Garmin GDL90 binary reporting protocol.

4.19.2 Query the Reporting Protocol for All Serial Interfaces

```
AT+PROTOCOL_OUT?
PROTOCOL_OUT=CONSOLE,<console_protocol>
PROTOCOL_OUT=COMMS_UART,<comms_uart_protocol>
```

Queries the reporting protocol set for each of the serial interfaces. See the table in 4.19.1 for protocol definitions.

4.20 AT+REBOOT

```
AT+REBOOT
```

Reboots the ADSBee 1090 immediately.

4.21 AT+RX_ENABLE

4.21.1 Enable or Disable the Receiver(s)

```
AT+RX_ENABLE=<all_enabled>,<1090_enabled>,<subg_enabled>
```

Parameter	Description	Values
all_enabled	Flag indicating whether to ignore 1090_enabled and subg_enabled flags and enable / disable all radios.	1 – All radios enabled. 0 – All radios disabled.
1090_enabled	Flag indicating whether the 1090MHz receiver should be enabled or disabled. NOTE: Disabling the 1090MHz receiver does not conserve any power, as the RF frontend is still active. This receiver disable flag simply disables an interrupt within the RP2040 that is used to check for incoming transponder messages.	1 – 1090MHz receiver is enabled. 0 – 1090MHz receiver is disabled.
subg_enabled	Flag indicating whether the Sub-GHz receiver should be enabled or disabled (if equipped). Disabling the Sub-GHz receiver does save a small amount of power, as the Sub-GHz transceiver chip will be powered down.	1 – Sub-GHz receiver is enabled. 0 – Sub-GHz receiver is disabled.

4.21.2 Check Whether the Receiver(s) are Currently Enabled

```
AT+RX_ENABLE?
1090 Receiver: ENABLED
SubG Receiver: DISABLED
```

4.22 AT+RX_POSITION

ADSBee's internal receiver position is used as a reference location for decoding Mode S surface position packets, and is also sent as an ownship location in various packet formats, enabling electronic flight bag apps to "scroll" their maps in order to follow the location of the receiver.

The receiver location can be hardcoded to a fixed value or can be automatically derived using a number of methods.

Supported position sources:

- NONE – No position set nor available.
- FIXED – Receiver position set to a fixed location.
- GNSS – Receiver position derived from a connected GNSS receiver.
- LOWEST – Receiver position derived from the lowest altitude aircraft being tracked in the aircraft dictionary. Generally aircraft lower to the ground are harder to receive, so the lowest aircraft provides a good guess of where the receiver might be located. **This is the default setting.**
- ICAO – Receiver position tracks the ICAO address of a specific aircraft. This is useful for bootstrapping ownship position from an aircraft that the receiver might be on board or following.

4.22.1 Query Receiver Position

```
AT+RX_POSITION?
Receiver Position:
        Source: LOWEST [OK]
        Latitude: 54.313381 deg
        Longitude: -170.856537 deg
        GNSS Altitude: 36325 ft
        Barometric Altitude: 37000 ft
        Heading: 277.5 deg
        Speed: 500 kts
        ICAO: 0xAA92F9 [NOT IN USE]
```

The Position Available field (square brackets to the right of Source) indicates whether the requested receiver position source is providing valid data. For instance, if a receiver position source is set to ICAO, but an aircraft with that ICAO is not found, the Position Available field will show `[NOT AVAILABLE]` when an aircraft with the requested ICAO is found, the field will show `[OK]`. The same behavior applies when the receiver position is set to LOWEST (ie. The position valid field only shows `[OK]` if there is at least one aircraft with a valid altitude in the aircraft dictionary.

The ICAO In Use field (square brackets to the right of ICAO) indicates whether the stored ICAO address is actively in use. This will show `[IN USE]` when the receiver has its source set to ICAO, and will show `[NOT IN USE]` otherwise.

4.22.2.1 Disable Receiver Position

```
AT+RX_POSITION=NONE
OK
```

4.22.2.2 Set Receiver Position to Fixed Value

```
AT+RX_POSITION=FIXED,<lat_deg>,<lon_deg>,<gnss_alt_ft>,<baro_alt_ft>,<heading_deg>
OK
```

Parameter	Description	Values
lat_deg	Latitude of the receiver in degrees.	-90.0 to 90.0
lon_deg	Longitude of the receiver in degrees.	-180.0 to 180.0
gnss_alt_ft	GNSS (WGS84) altitude of the receiver, in feet.	Integer
baro_alt_ft	Barometric altitude of the receiver, in feet.	Integer
heading_deg	Heading of the receiver, in degrees.	0 to 359.99

4.22.2.3 Set Receiver Position to GNSS Receiver Position

```
AT+RX_POSITION=GNSS
OK
```

Note: This is not yet implemented.

4.22.2.4 Set Receiver Position to Track Lowest Aircraft

```
AT+RX_POSITION=LOWEST
OK
```

4.22.2.5 Set Receiver Position to Track Aircraft by ICAO

```
AT+RX_POSITION=ICAO,<icao_address>
OK
```

Parameter	Description	Values
icao_address	Hexadecimal formatted 24-bit ICAO address of the aircraft to track. Do not include a leading 0x (just the hexadecimal digits). If the aircraft ICAO is present in multiple aircraft dictionary entries (e.g. the aircraft is transmitting on Mode S, and is being re-broadcast on UAT), the receiver will use the position of the aircraft entry that had a packet received most recently.	AA92F9

4.23 AT+SETTINGS

4.23.1 Query Current Settings

```
AT+SETTINGS?
Settings Struct
        Receiver: ENABLED
        Trigger Level: 1300 milliVolts (-62 dBm)
        Bias Tee: DISABLED
        Watchdog Timeout: 10 seconds
        Log Level: WARNINGS
        Reporting Protocols:
                CONSOLE: NONE
                COMMS_UART: CSBEE
        Comms UART Baud Rate: 115207 baud
        GNSS UART Baud Rate: 9600 baud
        ESP32: ENABLED
        WiFi AP: ENABLED
                Channel: 3
                SSID: ADSBee1090-0240907000001
                Password: yummyflowers
        WiFi STA: ENABLED
                SSID: Smart Bidet Wifi
                Password: ********
        Feed URIs:
                0 URI:192.168.1.103 Port:30004 ACTIVE Protocol:BEAST
ID:0xbee00038172d18c1
                1 URI: Port:0 INACTIVE Protocol:NONE ID:0xbee00038172d18c1
                2 URI: Port:0 INACTIVE Protocol:NONE ID:0xbee00038172d18c1
                3 URI: Port:0 INACTIVE Protocol:NONE ID:0xbee00038172d18c1
                4 URI:78.46.234.18 Port:30004 ACTIVE Protocol:BEAST ID:0xbee00038172d18c1
                5 URI:feed.whereplane.xyz Port:30004 INACTIVE Protocol:BEAST
ID:0xbee00038172d18c1
OK
```

4.23.2 Dump Settings in AT Format

Sending AT+SETTINGS?DUMP outputs a dump of all nonvolatile settings in AT command format, allowing them to be easily re-entered when restoring the ADSBee settings after a firmware upgrade, or for moving setting between devices.

```
AT+SETTINGS?DUMP
AT+SETTINGS=RESET
AT+BAUD_RATE=CONSOLE,115200
AT+BAUD_RATE=COMMS_UART,115200
AT+BAUD_RATE=GNSS_UART,115200
AT+BIAS_TEE_ENABLE=0
AT+ESP32_ENABLE=1
AT+FEED=0,,0,0,NONE
AT+FEED=1,,0,0,NONE
AT+FEED=2,,0,0,NONE
AT+FEED=3,,0,0,NONE
AT+FEED=4,feed.airplanes.live,30004,1,BEAST
AT+FEED=5,feed.adsb.fi,30004,1,BEAST
AT+LOG_LEVEL=WARNINGS
AT+PROTOCOL=CONSOLE,NONE
AT+PROTOCOL=COMMS_UART,MAVLINK1
AT+RX_ENABLE=1
AT+TL_SET=1300
AT+WATCHDOG=10
AT+WIFI_AP=1,ADSBee1090-0240907000001,yummyflowers,11
AT+WIFI_STA=0,,
OK
```

4.23.3 Load Settings from EEPROM

```
AT+SETTINGS=LOAD
```

4.23.4 Save Settings to EEPROM

```
AT+SETTINGS=SAVE
```

Note that this command is also used to synchronize settings between the RP2040 and ESP32. When changing parameters in the Serial CLI or Web CLI, run `AT+SETTINGS=SAVE` or reboot the device to synchronize new settings (e.g. new feed parameters or network settings).

4.23.5 Reset Settings to Factory Default

```
AT+SETTINGS=RESET
```

Note that this command resets settings to factory defaults, but will not persist them to the next power on unless `AT+SETTINGS=SAVE` is run subsequently.

4.24 AT+SUBG_ENABLE

Enable or disable the Sub-GHz receiver, if present. Can be used to drive the Sub-GHz receiver enable line HI or LO, or leave the corresponding GPIO in a high impedance state in order to allow external control (e.g. for using an external debugger).

4.24.1 Set Sub-GHz Receiver Enable Status and Line Drive Type

```
AT+SUBG_ENABLE=<enabled>
```

Parameter	Description	Values
enabled	Flag indicating whether the Sub-GHz receiver should be enabled or disabled.	0 – Sub-GHz receiver is disabled. 1 – Sub-GHz receiver is enabled. EXTERNAL – Sub-GHz receiver enable is pulled low by default, but can be overridden by an external device.

4.24.2 Query Sub-GHz Receiver Enable Status and Line Drive Type

```
AT+SUBG_ENABLE?
SUBG_ENABLE=1
```

4.25 AT+TEST

Used for running internal self-tests. Can only be run once per boot cycle.

```
AT+TEST
[==========] Running 4 test cases.
[ RUN      ] SpiCoprocessor.WriteReadScratchNoAck
[       OK ] SpiCoprocessor.WriteReadScratchNoAck (687000ns)
[ RUN      ] SpiCoprocessor.WriteReadScratchWithAck
[       OK ] SpiCoprocessor.WriteReadScratchWithAck (751000ns)
[ RUN      ] SPICoprocessor.ReadWriteReadRewriteRereadBigNoAck
[       OK ] SPICoprocessor.ReadWriteReadRewriteRereadBigNoAck (25336000ns)
[ RUN      ] SPICoprocessor.ReadWriteReadRewriteRereadBigWithAck
[       OK ] SPICoprocessor.ReadWriteReadRewriteRereadBigWithAck (29287000ns)
[==========] 4 test cases ran.
[  PASSED  ] 4 tests.
```

4.26 AT+TL_READ

```
AT+TL_READ?
TL_READ=1258mV (-64 dBm)
```

Queries the current value of the internal trigger level of the RF frontend's data slicer circuit. Signals peaking above this level will trigger a demodulation. This value should be set substantially above the noise floor in order to avoid false triggers.

Note that the value returned from TL_READ is different from querying TL_SET, since the value returned by TL_READ is read from the RP2040's ADC, while the value returned from querying TL_SET is the setpoint for the PWM output that generates the trigger level signal. They should be roughly similar, but slightly offset.

4.27 AT+TL_OFFSET

4.27.1 Set Trigger Level

```
AT+TL_OFFSET=<command>
```

Parameter	Description	Values
command	Trigger level offset (milliVolts above the measured noise floor), or the activation word to begin a self-learning process for the trigger level using simulated annealing.	0-3300 LEARN Trigger level defaults to 200.

The `AT+TL_OFFSET` command sets the value of the internal trigger level of the RF frontend's data slicer circuit. Signals peaking above this level will trigger a demodulation. This value should be set above the noise floor in order to avoid false triggers.

A higher trigger level makes the receiver less sensitive, while a lower trigger level makes the receiver more sensitive (and thus more likely to be triggered by unwanted noise).

A self-learning process can be activated by sending `AT+TL_OFFSET=LEARN` instead of providing a trigger value in milliVolts. The self-learning process uses simulated annealing, where a random trigger value is chosen, and then random neighbors are traversed to if their trigger levels seem to provide an improved number of valid packets within the learning window. The allowable traversal distance is gradually decreased over time until a final value is settled upon. The self-learning process takes approximately 20 minutes. Note that as of the current firmware revision, hand-tuning the trigger value (or using the provided default value) may outperform the self-learning routine.

4.27.2 Query Trigger Level

```
AT+TL_OFFSET?
TL_OFFSET=200mV
```

Queries the trigger level that is set internally. Note that this may differ from the actual value that is read using an ADC, as provided by `AT+TL_READ`.

4.28 AT+UPTIME

Queries the uptime of the ADSBee, in seconds.

```
AT+UPTIME?
UPTIME=71
```

4.29 AT+WATCHDOG

The ADSBee features a watchdog timer on the RP2040, which will automatically reset all microcontrollers on the board if code executing on the RP2040 freezes up, or if SPI communication with the ESP32 breaks down. This is intended to provide a means of automatic recovery in case of a hard fault caused by firmware bugs (likely), power glitches (maybe), or cosmic rays (probably not). As the watchdog may not be wanted in some cases, like debugging hardfaults, or during device programming / beakpoint debugging, it can be disabled. Additionally, means are provided for self-testing the watchdog via AT command in order to verify that it is still active.

4.29.1 Configure the Watchdog Timer

```
AT+WATCHDOG=<timeout_sec>
```

Parameter	Description	Values
command	Trigger level, in milliVolts, or the activation word to begin a self-learning process for the trigger level using simulated annealing.	0-3300 LEARN

4.29.2 Query the Watchdog Timer

```
AT+WATCHDOG?
WATCHDOG=10
```

Prints the value of the watchdog timer in seconds, or 0 if the watchdog timer is disabled.

4.29.3 Test the Watchdog Timer

```
AT+WATCHDOG=TEST
Blocking for 11 seconds.
```

(ADSBee Reboots here)

The AT+WATCHDOG=TEST command blocks the processor for the currently configured watchdog timeout plus one second. This should induce a device reboot, otherwise an error message will be displayed indicating that the watchdog failed to activate.

4.30 AT+WIFI_AP

The ADSBee creates its own WiFi access point that allows other devices to join its network in order to access the Web CLI and receive aircraft data (e.g. GDL90 traffic info over UDP port 4000). The WiFi AP can be enabled/disabled and can have its SSID, password, and WiFi channel customized. Note that WiFi settings don't have any affect unless the ESP32 is enabled, and are not propagated to the ESP32 unless settings are saved with `AT+SETTINGS=SAVE`.

4.30.1 Configure the WiFi Access Point

```
AT+WIFI_AP=<enabled>,<ap_ssid>,<ap_pwd>,<ap_channel>
```

Parameter	Description	Values
enabled	Toggle whether the AP is enabled or disabled.	0 – Disable WiFi access point. 1 – Enable WiFi access point.
ap_ssid	SSID for the access point. Defaults to a unique SSID with the format "ADSBee1090-0240907000001". Can be set to a value up to 32 characters long.	ASCII string up to 32 characters in length.
ap_pwd	Password for the access point. Defaults to "yummyflowers". Can be set to a value up to 64 characters long.	ASCII string up to 64 characters in length.
ap_channel	WiFi channel number for the access point. Can be set to a value from 1-11. A random channel value is chosen by default during initial programming of the device. It is recommended to change the channel value to a frequency with low utilization in the installation location. Note that the ap_channel is overridden automatically if both the access point and station mode are used simultaneously; the wifi access point will use only the channel that was selected by the wifi station when connecting to a network.	1-11

4.30.2 Query the WiFi Access Point Configuration

```
AT+WIFI_AP?
WIFI_AP=1,ADSBee1090-0240907000001,yummyflowers,3
```

Reply format matches argument values and ordering from the configuration command.

4.31 AT+WIFI_STA

The ADSBee can join external WiFi networks as a WiFi station in order to allow connections from other devices on the existing network, or to connect to the internet in order to feed remote endpoints with air traffic data.

4.31.1 Configure the WiFi Station

```
AT+WIFI_STA=<enabled>,<sta_ssid>,<sta_pwd>
```

Parameter	Description	Values
enabled	Toggle whether the AP is enabled or disabled.	0 – Disable WiFi station. 1 – Enable WiFi station.
sta_ssid	SSID for the access point to connect to. Can be set to a value up to 32 characters long.	ASCII string up to 32 characters in length.
sta_pwd	Password for the access point to connect to. Can be set to a value up to 64 characters long.	ASCII string up to 64 characters in length.

4.31.2 Query the WiFi Station Configuration

```
WIFI_STA=1,My Supercool WiFi Network,**********
```

Queries the configuration used by the ADSBee to join an external WiFi network. Note that the value and ordering of the arguments is identical to the configuration command, except the password is censored.

5 Reporting Protocols

ADSBee 1090 supports the following reporting protocols on CONSOLE and COMMS_UART.

- CSBee
- GDL90
- MAVLINK 1
- MAVLINK 2
- Mode S Beast
- Raw Packets

The CONSOLE interface reports debug messages and AT command responses in addition to the selected reporting protocol. If the CONSOLE interface is being used as a reporting interface, it is recommended to send AT+LOG_LEVEL=SILENT to silence any debug logs that might corrupt the reported data, and to avoid sending additional AT commands while reading reported data in order to avoid the reported data being interspersed with OK and other AT command responses from the ADSBee 1090.

5.1 CSBee

Comma Separated Bee protocol containing information about tracked aircraft as plain text.

The CSBee protocol is heavily inspired by the Aerobits Aero CSV protocol.

5.1.1 CSBee Protocol Version History

Consult the table below for a version history of the CSBee protocol. Each version of the ADSBee firmware writes only the most recent version of the CSBee protocol. All versions of the CSBee protocol except for the most recent version are considered deprecated. Whenever possible, efforts are made to reduce breaking changes to the CSBee protocol when incrementing the version number.

Current protocol version: 3

CSBee Protocol Version	Introductory Firmware Revision	Description
1		Initial release.
2	adsbee_1090-0.9.0-rc1	• Add UAT contacts. • Change bit flag positions. • Add dedicated fields for GNSS / baro vertical rates. • Add additional aircraft info (e.g. dimensions).
3	adsbee_1090-0.9.0-rc15	• Add is_non_transponder flag to Mode S aircraft. • Prepend address qualifier to UAT ICAO addresses. • Add is_utc_coupled flag to UAT aircraft.

5.1.2 Mode S Aircraft Message

This message contains information about an aircraft being tracked via ADS-B (1090MHz). Aircraft reports are provided once per second, per aircraft, until contact with the aircraft has been lost for 60 seconds.

```
#A:ICAO,FLAGS,CALL,SQUAWK,ECAT,LAT,LON,BARO_ALT,GNSS_ALT,DIR,SPEED,BARO_VRATE,GNSS_VRATE,
NICNAC,ACDIMS,VERSION,SIGS,SIGQ,SFPS,ESFPS,CRC\r\n
```

Field	Description	Format	Example value
#A	Mode S aircraft message start indicator.		
ICAO	ICAO number of aircraft (3 bytes).	Hex Integer	3C65AC
FLAGS	Flags bitfield, see table 5.1.2.1.	Hex Integer	12F356A8
CALL	Callsign of aircraft.	String	N61ZP
SQUAWK	SQUAWK of aircraft.	Octal Integer	7232
ECAT	Emitter category, see table 5.1.2.2.	Integer	14
LAT	Latitude, in degrees.	Float	57.57634
LON	Longitude, in degrees.	Float	17.59554
BARO_ALT	Barometric altitude, in feet.	Integer	5000
GNSS_ALT	Geometric altitude, in feet.	Integer	5000
DIR	Direction of aircraft, in degrees [0,360). Consult bit flags to determine whether this field is true heading, magnetic heading, or track.	Integer	35
SPEED	Horizontal velocity of aircraft, in knots.	Integer	464
BARO_VRATE	Barometric vertical velocity of aircraft, in ft/min.	Integer	-1344
GNSS_VRATE	Geometric vertical velocity of the aircraft, in ft/min	Integer	-1344
NICNAC	Aircraft data integrity. See table 5.1.2.3	Hex Integer	
ACDIMS	Aircraft physical dimensions and system design assurance. See table 5.1.2.4.	Hex Integer	31BE89F2
VERSION	ADS-B protocol version used by the aircraft.	Integer	2
SIGS	Signal strength, in dBm.	Integer	-92
SIGQ	Signal quality, in dB.	Integer	2
SFPS	Number of valid 56-bit Squitter Mode S frames received from the aircraft during the last second.	Integer	2
ESFPS	Number of valid 112-bit Extended Squitter Mode S frames received from the aircraft during the last second.	Integer	5
CRC	CRC16 (described in 5.1.2.5).	Hex Integer	2D3E

5.1.2.1 FLAGS Bitfield

Note: All bits 25-31 are momentary (cleared and updated every reporting interval).

Bit	Bit Name	Meaning if the bit is set (1)
0	IS_AIRBORNE	Emitter is airborne.
1	BARO_ALTITUDE_VALID	Emitter has provided a barometer altitude.
2	GNSS_ALTITUDE_VALID	Emitter has provided a GNSS altitude.
3	POSITION_VALID	Emitter has provided a pair of even and odd Compact Position Reporting packets that were decoded to a location.
4	DIRECTION_VALID	Emitter has provided a direction (may be ground track, magnetic compass heading, or true compass heading).
5	HORIZONTAL_SPEED_VALID	Emitter has provided a horizontal velocity.
6	BARO_VERTICAL_RATE_VALID	Emitter has provided a barometric vertical velocity.
7	GNSS_VERTICAL_RATE_VALID	Emitter has provided a GNSS vertical velocity.
8	IS_MILITARY	Emitter has transmitted at least one packet using a military format, such as Military Extended Squitter (DF=19).
9	IS_CLASS_B2_GROUND_VEHICLE	Emitter is actually a ground vehicle using a Class B2 transponder with a transmission power < 70W.
10	HAS_1090_ES_IN	Emitter has receive capability for 1090MHz Extended Squitter transmissions.
11	HAS_UAT_IN	Emitter has receive capability for UAT (978MHz Universal Access Transceiver) transmissions.
12	TCAS_OPERATIONAL	Emitter has a functional TCAS (Traffic Collision Avoidance System) onboard.
13	SINGLE_ANTENNA	Emitter is using a single antenna, instead antennas above and below the fuselage. Transmissions may be weak or irregular during maneuvering.
14	DIRECTION_IS_HEADING	Surface position messages provided by the aircraft indicate a heading and not a track angle.
15	HEADING_USES_MAGNETIC_NORTH	Heading reported by the aircraft while on the surface uses magnetic north instead of true north.
16	IDENT	The aircraft has its SPI (Special Position Identification) bits set in Mode A/C or Mode S messages. This indicates that the pilot has depressed the momentary IDENT switch on their transponder, most likely at the request of air traffic control.
17	ALERT	The aircraft is issuing either a permanent or momentary alert. This could correspond to an operational mode change or something else.
18	TCAS_RA	The aircraft has an active TCAS resolution advisory (i.e. the aircraft is warning the pilot to take action in order to avoid colliding with another aircraft).
19	IS_NON_TRANSPONDER	This aircraft is being rebroadcast (not received from a transponder).
20-22 Reserved		
23	UPDATED_BARO_ALTITUDE	Barometric altitude has been updated within the last reporting interval.
24	UPDATED_GNSS_ALTITUDE	GNSS altitude has been updated within the last reporting interval.
25	UPDATED_POSITION	Position (latitude / longitude) has been updated within the last reporting interval.
26	UPDATED_DIRECTION	Direction has been updated within the last reporting interval.
27	UPDATED_HORIZONTAL_SPEED	Horizontal velocity has been updated within the last reporting interval.
28	UPDATED_BARO_VERTICAL_RATE	Barometric vertical velocity has been updated within the last reporting interval.
29	UPDATED_GNSS_VERTICAL_RATE	GNSS vertical velocity has been updated within the last reporting interval.
30-31 Reserved		

5.1.2.2 ECAT Field

The ECAT field indicates the Emitter Category (i.e. airframe type) for each ADSB emitter that is being tracked. This field contains information about what kind of aircraft, ground vehicle, obstacle, or other airspace user is emitting ADS-B packets, and can be used to understand the emitter's maneuvering capability and potential for wake vortex impact.

ECAT Value	Emitter Category
-1	Invalid
0	No Category Information
1	Light Aircraft (< 7,000kg)
2	Medium 1 (7,000kg – 34,000kg)
3	Medium 2 (34,000kg – 136,000kg)
4	High Vortex Aircraft
5	Heavy (> 136,000kg)
6	High Performance (> 5 G acceleration and > 400 kts speed)
7	Rotorcraft
8	Reserved
9	Glider / Sailplane
10	Lighter Than Air
11	Parachutist / Skydiver
12	Ultralight / Hang Glider / Paraglider
13	Reserved 1
14	Unmanned Aerial Vehicle
15	Space / Transatmospheric Vehicle
16	Reserved 2
17	Surface Emergency Vehicle
18	Surface Service Vehicle
19	Point Obstacle
20	Cluster Obstacle
21	Line Obstacle

5.1.2.3 NICNAC Bitfield

NICNAC Bitfield															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Source Integrity Level (SIL)		Geometric Vertical Accuracy (GVA)		Navigation Accuracy Category: Position (NAC_p)				Navigation Accuracy Category: Velocity (NAC_v)			Navigation Integrity Category: Barometer (NIC_{baro})	Navigation Integrity Category (NIC)			

NICNAC[0-3]: Navigation Integrity Category (NIC)

The radius of containment (NIC) indicates how much trust should be placed in an aircraft's reported location in the horizonal plane. The NIC reports a radius of containment specified by the avionics system of the emitter. The probability that the aircraft is outside of this radius of containment due to its avionics system receiving a faulty signal from one of its inputs (without displaying an error) is provided by another bitfield called the Source Integrity Level (SIL). Combined, the NIC and SIL indicate how likely it is that an aircraft is not actually contained by a bubble of a specified size, centered at the aircraft's reported location, assuming that the avionics onboard the aircraft are functioning correctly but may be given faulty inputs.

A higher NIC value indicates more trust in an aircraft's reported latitude / longitude position.

NIC Value	0	1	2	3	4	5	6	7	8	9	10	11
Radius of Containment	Unknown	< 20 NM	< 8 NM	< 4 NM	< 2 NM	< 1 NM	< 0.6 NM	< 0.2 NM	< 0.1 NM	< 75 m	< 25 m	< 7.5 m

NICNAC [4]: Navigation Integrity Category: Barometer (NIC_{baro})

The barometric altitude integrity (NIC_{baro}) indicates how much trust should be placed in an aircraft's reported altitude. The field is a single bit that indicates whether the aircraft uses an altimeter that has been cross-checked against other sources. Old school encoding altimeters have many parallel wires and output an altitude in a format called a Gillham Code, and have no built-in method of error checking. A single faulty wire can result in erroneous readings, so this bit lets air traffic control know whether to take altitude readings from the aircraft with a grain of salt. A 0 indicates that the transponder is outputting altitude from a Gillham coded source (with no way to cross check the value), while a 1 indicates that the transponder is outputting altitude from a Gillham coded source while using another sensor to cross-check it, or is using a more modern barometer that supports a protocol with built-in error checking.

A higher NIC_{baro} value (i.e. 1 instead of 0) indicates more trust in the aircraft's reported altitude.

NIC_{baro} Value	0	1
Barometric Altitude Integrity	Altitude is from a Gillham-coded input, not cross checked.	Altitude is from a Gillham-coded input that is being cross-checked with another source, or from a non Gillham-coded input with built-in error checking features.

NICNAC [5-7]: Navigation Accuracy Category: Velocity (NAC_v)

The horizontal velocity error (NAC_v) indicates the expected accuracy of the reported velocity of the aircraft when systems are operating nominally. This varies depending on the accuracy capabilities of the measurement equipment onboard the aircraft, and not how often we expect said equipment to fail.

A higher NAC_v indicates a more accurate velocity measurement system.

NAC_v Value	Horizontal Velocity Error
0b000	Unknown or ≥ 10 m/s
0b110	< 10 m/s
0b010	< 3 m/s
0b011	< 1 m/s
0b100	< 0.3 m/s

NICNAC [8-11]: Navigation Accuracy Category: Position (NAC_p)

The estimated position uncertainty (NAC_p) indicates the expected accuracy of the aircraft's reported location when systems are operating nominally. This varies depending on the capabilities of the aircraft's positioning system and not on how often we expect said equipment to fail.

A higher NAC_p indicates a more accurate positioning system.

NAC_p Value	0	1	2	3	4	5	6	7	8	9	10	11
Estimated Position Uncertainty	Unknown or ≥ 10NM	< 10 NM	< 4 NM	< 2 NM	< 1 NM	< 0.5 NM	< 0.3 NM	< 0.1 NM	< 0.5 NM	< 30 m	< 10 m	< 3 m

NICNAC[12-13]: Geometric Vertical Accuracy (GVA)

The geometric vertical accuracy indicates the 95% confidence interval (vertical figure of merit) provided by the aircraft's onboard GNSS system (i.e. assuming some distribution of altitudes, the aircraft's GNSS system is confident that 95 out of 100 times, the aircraft falls within some height of the reported geometric altitude.

GVA Value	0	1	2	3
95% Vertical Figure of Merit (VFOM)	Unknown or ≥ 150 m	< 150 m	≤ 45 m	< 45 m (Was previously "reserved", the actual value of this field may change but is guaranteed to be < 45 m).

NICNAC[14-15]: Source Integrity Level (SIL)

The source integrity level indicates the probability that the aircraft exceeds the bounds of its horizontal radius of containment (NIC) due to a silent fault in signals received by the aircraft (no avionics failure).

SIL Value	Probability of Exceeding NIC Radius of Containment Due to Silent Fault
0	Unknown or > $1x10^{-3}$ per flight hour.
1	≤ $1x10^{-3}$ per flight hour.
2	≤ $1x10^{-5}$ per flight hour.
3	≤ $1x10^{-7}$ per flight hour.
4	Unknown or > $1x10^{-3}$ per sample.
5	≤ $1x10^{-3}$ per sample.
6	≤ $1x10^{-5}$ per sample.
7	≤ $1x10^{-7}$ per sample.

5.1.2.4 ACDIMS Bitfield

ACDIMS Bitfield															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
GNSS Offset Forward of Reference Point (GOF)								GNSS Offset Right of Reference Point (GOR)							

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Aircraft Length (ACL)							Aircraft Width (ACW)							System Design Assurance (SDA)	

ACDIMS[0-1]: System Design Assurance (SDA)

The system design assurance indicates how robust the aircraft's position reporting systems are to failures of various severities. For instance, SDA = 1, a low SDA value, corresponds to Software and Hardware Design Assurance Level D, which states that a minor failure could cause the aircraft to transmit misleading position information with a probability of $\leq 1x10^{-3}$ per flight hour. A more robust system with SDA = 3, corresponding to Software and Hardware Design Assurance Level B, is expected to transmit misleading position information with a probability of $1x10^{-7}$ per flight hour even under a Hazardous failure condition.

Software Design Assurance categories used in this field are classified under RTCA DO-178B, Airborne Electronic Hardware Design Assurance are classified under RTCA DO-254, and failure classification levels are defined in FAA Advisory Circular AC-23.1309-1E.

SDA Value	Supported Failure Condition	Probability of Undetected Fault causing transmission of False or Misleading Information	Software and Hardware Design Assurance Level
0	Unknown / No Safety Effect	$> 1x10^{-3}$ per flight hour or unknown.	N/A
1	Minor	$\leq 1x10^{-3}$ per flight hour.	D
2	Major	$\leq 1x10^{-5}$ per flight hour.	C
3	Hazardous	$\leq 1x10^{-7}$ per flight hour.	B

ACDIMS[2-8] Aircraft Width (ACW)

Approximate value for the width of the aircraft bounding box, in meters. Can be interpreted as a 7-bit unsigned integer.

ACDIMS[9-15] Aircraft Length (ACL)

Approximate value for the length of the aircraft bounding box, in meters. Can be interpreted as a 7-bit unsigned integer.

ACDIMS[16-23] GNSS Offset Right of Reference Point (GOR)

Signed 8-bit integer representing the left/right offset of the GNSS antenna from the aircraft's reference point. A positive value indicates that the GNSS antenna is to the right of the aircraft reference point.

ACDIMS[24-31] GNSS Offset Forward of Reference Point (GOF)

Signed 8-bit integer representing the forward/back offset of the GNSS antenna from the aircraft's reference point. A positive value indicates that the GNSS antenna is forward of the aircraft's reference point. Always 0 for Mode S aircraft.

5.1.2.5 CRC Field

CSBee messages use a 16-bit Cyclical Redundancy Checksum (CRC-16), which can be calculated using the algorithm in the C++ code snippet below. Note the “swap16” helper function which also needs to be included.

```
uint16_t swap16(uint16_t value) { return (value << 8) | (value >> 8); }

uint16_t CalculateCRC16(const uint8_t *data_p, int32_t length) {
    uint8_t x;
    uint16_t crc = 0xFFFF;
    while (length--) {
        x = crc >> 8 ^ *data_p++;
        x ^= x >> 4;
        crc = (crc << 8) ^ ((uint16_t)(x << 12)) ^ ((uint16_t)(x << 5)) ^ ((uint16_t)x);
    }
    return swap16(crc);
}
```

5.1.3 UAT Aircraft Message

```
#U:ICAO,UAT_FLAGS,CALL,SQUAWK,ECAT,LAT,LON,BARO_ALT,GNSS_ALT,DIR,SPEED,BARO_VRATE,GNSS_VR
ATE,UAT_EMERG,NICNAC,ACDIMS,VERSION,SIGS,SIGQ,UATFPS,CRC\r\n
```

Field	Description	Format	Example value
#U	UAT aircraft message start indicator.		
ICAO	First Digit (optional): Address Qualifier (1 digit: see table 5.1.3.1) Last 6 Digits: ICAO address of aircraft (6 hex digits).	Hex Integer	13C65AC 3C65AC
UAT_FLAGS	Flags bitfield, see table 5.1.3.2.	Hex Integer	12F356A8
CALL	Callsign of aircraft.	String	N61ZP
SQUAWK	SQUAWK of aircraft.	Octal Integer	7232
ECAT	Emitter category, see table 5.1.2.2.	Integer	14
LAT	Latitude, in degrees.	Float	57.57634
LON	Longitude, in degrees.	Float	17.59554
BARO_ALT	Barometric altitude, in feet.	Integer	5000
GNSS_ALT	Geometric altitude, in feet.	Integer	5000
DIR	Direction of aircraft, in degrees [0,360). Consult bit flags to determine whether this field is true heading, magnetic heading, or track.	Integer	35
SPEED	Horizontal velocity of aircraft, in knots.	Integer	464
BARO_VRATE	Barometric vertical velocity of aircraft, in ft/min.	Integer	-1344
GNSS_VRATE	Geometric vertical velocity of the aircraft, in ft/min	Integer	-1344
UAT_EMERG	Aircraft UAT emergency / priority status. See table.	Integer	0
NICNAC	Aircraft data integrity. See table 5.1.2.3	Hex Integer	
ACDIMS	Aircraft physical dimensions and system design assurance. See table 5.1.2.4.	Hex Integer	31BE89F2
VERSION	UAT protocol version used by the aircraft.	Integer	1
SIGS	Signal strength, in dBm.	Integer	-92
SIGQ	Number of bits corrected using UAT forward error correction. 0 (best), 6 (worst).	Integer	2
UATFPS	Number of valid UAT frames received from the aircraft in the last second.	Integer	1
CRC	CRC16 (described in 5.1.2.5).	Hex Integer	2D3E

NOTE: See Mode S Aircraft message decoding reference for any bitfields not prefixed with “UAT_”.



5.1.3.1 UAT Address Qualifier

Address Qualifier Value	Meaning
0	Contact is ADSB target (broadcast by transponder) with 24-bit ICAO address.
1	Contact is ADSB target (broadcast by transponder) with self-assigned temporary address.
2	Contact is TIS-B target (broadcast by ground station), using a 24-bit ICAO address.
3	Contact is TIS-B target (broadcast by ground stations) with track file identifier.
4	Contact is a surface vehicle.
5	Contact is a fixed ADSB beacon.

5.1.3.2 UAT_FLAGS Bitfield

Bit	Bit Name	Meaning if the bit is set (1)
0	IS_AIRBORNE	Emitter is airborne.
1	BARO_ALTITUDE_VALID	Emitter has provided a barometer altitude.
2	GNSS_ALTITUDE_VALID	Emitter has provided a GNSS altitude.
3	POSITION_VALID	Emitter has provided a pair of even and odd Compact Position Reporting packets that were decoded to a location.
4	DIRECTION_VALID	Emitter has provided a direction (may be ground track, magnetic compass heading, or true compass heading).
5	HORIZONTAL_SPEED_VALID	Emitter has provided a horizontal velocity.
6	BARO_VERTICAL_RATE_VALID	Emitter has provided a barometric vertical velocity.
7	GNSS_VERTICAL_RATE_VALID	Emitter has provided a GNSS vertical velocity.
8	HAS_CDTI	Aircraft has Cockpit Display of Traffic Information (CDTI) equipped.
9	TCAS_OPERATIONAL	Aircraft is equipped with an operational TCAS / ACAS system.
10	DIRECTION_IS_HEADING	Surface position messages provided by the aircraft indicate a heading and not a track angle.
11	HEADING_USES_MAGNETIC_NORTH	Heading reported by the aircraft while on the surface uses magnetic north instead of true north.
12	IDENT	The aircraft has its SPI (Special Position Identification) bits set in Mode A/C or Mode S messages. This indicates that the pilot has depressed the momentary IDENT switch on their transponder, most likely at the request of air traffic control.
13	IS_UTC_COUPLED	The aircraft is transmitting frames precisely aligned to the UTC second.
14 Reserved		
15	TCAS_RA	The aircraft has an active TCAS resolution advisory (i.e. the aircraft is warning the pilot to take action in order to avoid colliding with another aircraft).
16	RECEIVING_ATC_SERVICES	
17-22 Reserved		
23	UPDATED_BARO_ALTITUDE	Barometric altitude has been updated within the last reporting interval.
24	UPDATED_GNSS_ALTITUDE	GNSS altitude has been updated within the last reporting interval.
25	UPDATED_POSITION	Position (latitude / longitude) has been updated within the last reporting interval.
26	UPDATED_DIRECTION	Direction has been updated within the last reporting interval.
27	UPDATED_HORIZONTAL_SPEED	Horizontal velocity has been updated within the last reporting interval.
28	UPDATED_BARO_VERTICAL_RATE	Barometric vertical velocity has been updated within the last reporting interval.
29	UPDATED_GNSS_VERTICAL_RATE	GNSS vertical velocity has been updated within the last reporting interval.
30-31 Reserved		



5.1.3.3 UAT_EMERG Bitfield

Contains the emergency / priority status reported by a UAT emitter.

UAT_EMERG Value	Meaning
0	No emergency.
1	General emergency.
2	Lifeguard / medical emergency.
3	Minimum fuel.
4	No communications.
5	Unlawful interference.
6	Downed aircraft.
7	Reserved.

5.1.4 Statistics Message

This message contains some useful statistics about ADSBee's operational status. Format of that frame is shown below:

```
#S:VERSION,DPS,RAW_SFPS,SFPS,RAW_ESFPS,ESFPS,RAW_UATFPS,UATFPS,TSCAL,UPTIME,CRC\r\n
```

Field	Description	Format	Example Value
#S	Statistics message start indicator.		
VERSION	Version of the CSBee protocol being used.	Integer	3
SDPS	Number of attempted Mode S demodulations in the last second.	Integer	106
RAW_SFPS	Number of squitter (56-bit Mode S) frames received in the last second.	Integer	150
SFPS	Number of valid squitter (56-bit Mode S) frames received in the last second.	Integer	20
RAW_ESFPS	Number of extended squitter (112-bit Mode S) frames received in the last second.	Integer	15
ESFPS	Number of valid Mode S frames received in the last second.	Integer	3
RAW_UATFPS	Number of UAT frames received in the last second.	Integer	5
UATFPS	Number of valid UAT frames received in the last second.	Integer	4
NUM_AIRCRAFT	Number of aircraft tracked in the last second.	Integer	20
TSCAL	Calibration value for TS field in raw frames	Integer	13999415
UPTIME	Receiver uptime, in seconds.	Integer	13456
CRC	CRC16 (described in 5.1.4.1).	Hex Integer	2D3E

5.1.4.1 CRC Field

See 5.1.2.5.

5.2 GDL90

ADSBee complies with the GDL90 public data interface specification 560-1058-00 Rev A.

5.3 MAVLINK

Tracked aircraft information is sent in MAVLINK ADSB_VEHICLE messages, in a data burst once per second. The data burst consists of 1x MAVLINK Heartbeat message, Nx ADSB_VEHICLE messages, where N is the number of tracked aircraft, and 1x special delimiter message (MAVLINK_REQUEST_DATA_STREAM for MAVLINK1 mode, MAVLINK_MESSAGE_INTERVAL for MAVLINK2 mode) which indicates the end of the list of tracked aircraft. Note that this is a binary protocol which is not human readable.

WARNING: Windows has a bug, which causes some machines to recognize a serial port reporting MAVLINK packets as a mouse, which can result in phantom mouse movements and clicks (ask me how I know). Please use caution while running MAVLINK on a serial port while the computer is unattended.

5.3.1 MAVLINK_HEARTBEAT (Message ID 0) Packet Definition

Field Name	Type	Units	Values	Description
custom_mode	`uint32_t`		0	Unused.
type	`uint8_t`		MAV_TYPE_ADSB	Always MAV_TYPE_ADSB (27).
autopilot	`uint8_t`		MAV_AUTOPILOT_INVALID	Always MAV_AUTOPILOT_INVALID (8).
base_mode	`uint8_t`		0	Unused.
system_status	`uint8_t`		MAV_STATE_ACTIVE	Always MAV_STATE_ACTIVE (4).
mavlink_version	`uint8_t`		MAVLINK_VERSION	1 for MAVLINK1 mode, 2 for MAVLINK2 mode.

5.3.2 MAVLINK ADSB_VEHICLE (Message ID 246) Packet Definition

From: https://mavlink.io/en/messages/common.html#ADSB_VEHICLE

Field Name	Type	Units	Values	Description
ICAO_address	`uint32_t`			ICAO address
lat	`int32_t`	degE7		Latitude
lon	`int32_t`	degE7		Longitude
altitude_type	`uint8_t`		ADSB_ALTITUDE_TYPE	ADSB altitude type.
altitude	`int32_t`	mm		Altitude(ASL)
heading	`uint16_t`	cdeg		Course over ground
hor_velocity	`uint16_t`	cm/s		The horizontal velocity
ver_velocity	`int16_t`	cm/s		The vertical velocity. Positive is up
callsign	`char[9]`			The callsign, 8+null
emitter_type	`uint8_t`		ADSB_EMITTER_TYPE	ADSB emitter type.
tslc	`uint8_t`	s		Time since last communication in seconds
flags	`uint16_t`		ADSB_FLAGS	Bitmap to indicate various statuses including valid data fields
squawk	`uint16_t`			Squawk code

5.3.3 MAVLINK_REQUEST_DATA_STREAM (Message ID 66) Packet Definition

This message serves as the delimiter which indicates the end of the aircraft list in MAVLINK1 mode.

Field Name	Type	Units	Description
req_message_rate	`uint16_t`		Always 0 (unused).
target_system	`uint8_t`		Always 0 (unused).
target_component	`uint8_t`		Always 0 (unused).
req_stream_id	`uint8_t`		Always 0 (unused).
start_stop	`uint8_t`		Always 0 (unused).

5.3.4 MAVLINK MESSAGE_INTERVAL (Message ID 244) Packet Definition

This message serves as the delimiter which indicates the end of the aircraft list in MAVLINK2 mode.

From: https://mavlink.io/en/messages/common.html#MESSAGE_INTERVAL

Field Name	Type	Units	Description
message_id	`uint16_t`		The ID of the requested MAVLink message. v1.0 is limited to 254 messages. NOTE: For ADSBee 1090, message_id is always 246, corresponding to the ADSB_VEHICLE message.
interval_us	`int32_t`	us	The interval between two messages. A value of -1 indicates this stream is disabled, 0 indicates it is not available, > 0 indicates the interval at which it is sent. NOTE: For ADSBee 1090, message_id is always 1000 us.

5.4 Mode S Beast

Mode S Beast is a popular feed protocol used by both open-source and commercial data exchanges to send Mode S data, often over TCP sockets or serial connections.

Mode S Beast sends packets as raw binary, with 0x1a used as an "escape" character indicating the start of a Beast frame. Any non start-of-frame instance of 0x1a is escaped with an additional 0x1a beforehand. Thus, 0x1a can be interpreted as the start of a Beast frame, and 0x1a1a can be interpreted as a 0x1a value within a Beast frame.

5.4.1 Beast: Mode S Frames

Field	Frame Start	Frame Type	Timestamp	RSSI	Payload
Length [Bytes]	1	1	6-12	1-2	Mode S Short: 7-14 Mode S Long: 14-28
Example	0x1a	0x32	0x123456781a1aff	0x56	<data>
Definition	0x1a Frame start is always an escape char.	0x32: Mode S Short 0x33: Mode S Long No escapes.	6-byte 12MHz MLAT timestamp, MSB first. May have up to 6 escapes.	RSSI byte is "Beast Power Level", which is the square root of the un-logged dBFS value. May have up to 1 escape.	Packet contents. May theoretically have up to <packet_length> escapes, but this would not correspond to a valid packet and will not be seen in practice.

5.4.2 Beast: UAT Frames

Field	Frame Start	Frame Type	UAT Message Type	Timestamp	RSSI	Payload
Length [Bytes]	1	1	1	6-12	1-2	UAT Short ADSB:
Example	0x1a	0xec	0x73	0x123456781a1aff	0x56	<data>
Definition	0x1a Frame start is always an escape char.	0xec Indicates frame type UAT (both ADSB and uplink). No escapes.	0x73 ('s'): UAT short ADSB (30 byte payload, not including escapes). 0x5c ('l'): UAT long ADSB (48 byte payload, not including escapes) 0x75 ('u'): UAT Uplink (552 byte payload, not including escapes) No escapes.	6-byte 12MHz MLAT timestamp, MSB first. May have up to 6 escapes.	RSSI byte is "Beast Power Level", which is the square root of the un-logged dBFS value. May have up to 1 escape.	Packet contents, including both payload and FEC parity. Uplink packets remain interleaved (transmitted byte order retained). May theoretically have up to <packet_length> escapes, but this would not correspond to a valid packet and will not be seen in practice.

5.5 Raw Packets

5.5.1 Raw Mode S Frames

Raw Mode S packets are reported in the format below. Only packets that have passed checksum validation and match a supported Mode S downlink format are reported.

```
#MDS*RAW_FRAME;(SOURCE,SIGS,SIGQ,TS)\r\n
```

Field	Description	Format	Example value
#MDS	Indicates the start of a raw Mode S frame.		
RAW_FRAME	The packet contents as raw hexadecimal. Varies in length depending on packet length.	Hexadecimal String	8D48C22D60AB0452BFAD19A695E0
SOURCE	An unsigned integer	`int16_t`	2
SIGS	Signal strength of the packet in dBm, printed as a decimal value.	`int16_t`	-60
SIGQ	Signal quality of the packet in dB, printed as a decimal value.	`int16_t`	2
TS	64-bit 48MHz MLAT counter ticks, printed as a hexadecimal string.	`uint64_t`	00000000FB671342

5.5.2 Raw UAT ADSB Frames

Raw UAT frames are reported in the format below. Only packets that have passed checksum validation are reported. Note that UAT uplink frames can be quite long (552 Bytes / 1104 chars), so serial buffers need to be able to handle this!

```
#UAT*-RAW_FRAME;(SIGS,SIGQ,TS)\r\n
```

Field	Description	Format	Example value
#UAT	Indicates the start of a raw UAT frame.		
RAW_FRAME	The packet contents as raw hexadecimal. Varies in length depending on packet length.	Hexadecimal String	
SIGS	Signal strength of the packet in dBm.	`int16_t`	-60
SIGQ	Signal quality of the packet in dB.	`int16_t`	2
TS	64-bit MLAT counter value in hexadecimal format.	`uint64_t`	00000000FB671342

5.5.3 Raw UAT Uplink Frames

Raw UAT uplink frames are reported in the format below. Only packets that have passed checksum validation are reported. Note that UAT uplink frames can be quite long (552 Bytes / 1104 chars), so serial buffers need to be able to handle this!

```
#UAT*+RAW_FRAME;(SIGS,SIGQ,TS)\r\n
```

Field	Description	Format	Example value
#UAT	Indicates the start of a raw UAT frame.		
RAW_FRAME	The packet contents as raw hexadecimal. Varies in length depending on packet length.	Hexadecimal String	
SIGS	Signal strength of the packet in dBm.	`int16_t`	-60
SIGQ	Signal quality of the packet in dB.	`int16_t`	2
TS	64-bit MLAT counter value in hexadecimal format.	`uint64_t`	00000000FB671342

5.6 Aircraft JSON WebSocket

The ADSBee 1090 exposes a WebSocket endpoint at ws://<device-ip>/aircraft that streams real-time aircraft data once per second. Each update consists of one JSON object per tracked aircraft, with objects separated by newline characters. The endpoint requires no authentication and supports up to 4 simultaneous connections. Connections that are idle for 90 seconds are automatically closed.

The ADSBee Aircraft JSON format is intended to be compatible with the JSON format emitted by readsb (https://github.com/wiedehopf/readsb/blob/dev/README-json.md).

Parameter	Value	Description
URL	/aircraft	WebSocket URI path
Protocol	WebSocket (RFC 6455)	HTTP upgrade required
Update Rate	1 Hz	Full aircraft list sent every 1000 ms
Max Clients	4	Simultaneous connections per device
Inactivity Timeout	90 s	Connection closed if no data exchanged
Max Message Size	4000 bytes	Per WebSocket frame

5.6.1 Message Format

Each WebSocket message contains one or more newline-terminated JSON objects. Each object represents a single tracked aircraft. Fields are omitted when their data has not been received or is not applicable to the aircraft type; clients must treat absent fields as unknown rather than zero.


```
{"hex":"a1b2c3","type":"adsb_icao","flight":"UAL123
","lat":37.62345,"lon":-122.37812,"alt_baro":37000,"gs":480,"track":270.0,"baro_rate":0,"
nic":8,"nic_baro":1,"nac_p":9,"nac_v":2,"sil":3,"gva":2,"sda":2,"version":2,"rssi":-68,"m
essages":142}
{"hex":"c3d4e5","type":"adsb_icao","on_ground":1,"lat":37.61200,"lon":-122.38500,"alt_bar
o":0,"gs":12,"track":90.0,"nic":8,"nic_baro":1,"nac_p":9,"nac_v":2,"sil":3,"gva":2,"sda":
2,"version":2,"rssi":-71,"messages":23}
```



5.6.2 JSON Fields

Field	Type	Condition	Mode S	UAT	Units	Description
hex	string	Always	✓	✓	—	6-digit ICAO 24-bit address in lowercase hex. UAT non-ICAO addresses (self-assigned or TIS-B track file) are prefixed with ~.
type	string	Always	✓	✓	—	Aircraft type. Mode S: adsb_icao, adsr_icao (non-transponder), adsb_icao_nt (Class B ground vehicle). UAT: adsb_icao, adsb_other, tisb_icao, tisb_trackfile.
flight	string	If callsign received	✓	✓	—	8-character flight identifier / callsign, right-padded with spaces.
squawk	string	If squawk received	✓	✓	—	4-digit octal transponder squawk code.
category	string	If category received	✓	✓	—	Emitter category in readsb A0-D7 format. Letter A-D = set 0-3; digit 0-7 = subcategory.
lat	number	If position valid	✓	✓	degrees	WGS-84 latitude, 5 decimal places (~1.1 m resolution).
lon	number	If position valid	✓	✓	degrees	WGS-84 longitude, 5 decimal places (~1.1 m resolution).
alt_baro	integer	If baro altitude valid	✓	✓	ft	Barometric pressure altitude.
alt_geom	integer	If GNSS altitude valid	✓	✓	ft	Geometric (GNSS/GPS) altitude.
gs	integer	If speed valid	✓	✓	kt	Ground speed.
track	number	If direction valid, not heading	✓	✓	degrees	True track over ground, 1 decimal place.
mag_heading	number	If heading valid, magnetic	✓	✓	degrees	Magnetic heading, 1 decimal place.
true_heading	number	If heading valid, true north	✓	✓	degrees	True heading, 1 decimal place.
baro_rate	integer	If baro vertical rate valid	✓	✓	ft/min	Barometric vertical rate of climb/descent.
geom_rate	integer	If GNSS vertical rate	✓	✓	ft/min	Geometric (GNSS) vertical rate of

		valid				climb/descent.
nic	integer	Always	✓	✓	—	Navigation Integrity Category (0-11).
nic_baro	integer	Always	✓	✓	—	Navigation Integrity Category Barometric (0-4).
nac_p	integer	Always	✓	✓	—	Navigation Accuracy Category - Position (0-11).
nac_v	integer	Always	✓	✓	—	Navigation Accuracy Category - Velocity (0-4).
sil	integer	Always	✓	✓	—	Surveillance Integrity Level (0-3).
gva	integer	Always	✓	✓	—	Geometric Vertical Accuracy (0-2).
sda	integer	Always	✓	✓	—	System Design Assurance (0-3).
version	integer	Always (Mode S); if >= 0 (UAT)	✓	✓	—	ADS-B version number.
rssi	integer	Always	✓	✓	dBm	Last received signal strength.
messages	integer	Always	✓	✓	—	Total valid messages received from this aircraft since tracking began.
alert	integer (1)	If alert flag set	✓	✗	—	Present (value 1) when the Mode S alert flag is active. Mode S only.
spi	integer (1)	If SPI flag set	✓	✗	—	Present (value 1) when the aircraft is squawking ident (Special Position Identification). Mode S only.
emergency	string	If emergency status set	✗	✓	—	Emergency/priority status: none, general, lifeguard, minfuel, nordo, unlawful, downed, reserved. UAT only.
on_ground	integer (1)	If NOT airborne	✓	✓	—	Present (value 1) when aircraft is on the ground. For UAT, also set for surface vehicles.